

Technical Development Bible

End-to-end engineering specification, reference code, dependency documentation, build plan and 3-person team execution plan

Version: 1.1 | Prepared: 15 August 2026 | Mobile stack: Flutter/Dart | Deployment target: Android-first hackathon MVP leading to controlled field pilot

Product contract: offline-first emergency communication using a Flutter mobile application on participating BLE-capable phones and fixed relay/beacon nodes; scoped Rooms; Mesh Code/QR join; local Speech-to-Text; short compressed voice-note store-and-forward over BLE; on-device incident triage; localization/density inputs; and an authority-facing local dashboard. The hackathon build targets Android physical devices first, while the application architecture remains Flutter/Dart and keeps platform-specific code behind narrow adapters.

IMPORTANT: There is no Text-to-Speech feature. Voice is captured by the sender. Speech-to-Text runs locally. The original short voice clip is compressed, chunked, relayed over BLE, reassembled and played by an authorized receiver. The design is explicitly not live voice streaming.

The previously mentioned “fourth feature” remains unspecified by the product team. This document does not invent it. A clean extension point is reserved for it.

Table of contents

1. Technical scope and requirements freeze
2. Engineering decisions that make the hackathon build feasible
3. End-to-end system architecture
4. Repository and module architecture
5. Toolchain, packages and official documentation
6. Core data contracts and Protocol Buffers
7. BLE discovery, GATT transport and phone-to-phone relay
8. Store-and-forward mesh protocol
9. Mesh Code / QR provisioning
10. Rooms and operational message routing
11. Voice capture, compression, packetization and playback
12. Offline Speech-to-Text architecture and inference plan
13. On-device AI triage and structured SOS
14. Localization, density and zone precursor scoring
15. Gateway and local control-room dashboard
16. Security, privacy and key management
17. Persistence, background execution and observability
18. Testing, benchmarking and failure injection
19. Complete hackathon build plan
20. Three-person team split and integration contracts
21. Demo runbook and definition of done
22. Production hardening roadmap
- Appendix A. Reference code snippets
- Appendix B. Command and debugging cheat sheet
- Appendix C. Official documentation index

1. Technical scope and requirements freeze

1.1 What must exist in the hackathon MVP

- Flutter mobile app installed on at least 3 physical BLE-capable Android phones, with 4-5 preferred for the final demo. The UI, state, persistence, event logic, voice workflow and transport orchestration are written in Dart/Flutter.
- Internet/cellular can be disabled and a structured SOS still travels from Phone A to a gateway/receiver through at least one intermediate phone.
- Mesh Code: user enters a short code or scans a QR to load the event/site manifest and allowed Rooms. The human code is never treated as the cryptographic root key.
- Rooms: at minimum Public Alerts, a zone Room, Medical and Responders. Room traffic is lower priority than SOS.
- Voice SOS: sender records a short clip; local STT produces a transcript; the original audio is compressed; transcript/structured SOS is relayed first; voice bytes follow as bounded chunks; receiver reassembles and can play the clip.
- On-device triage: raw typed/STT text becomes a schema-constrained Structured SOS with incident type, priority, hazards, confidence and rationale. Model failure never suppresses a manual SOS.
- Local persistence: pending messages survive process restarts long enough to retry; duplicate messages are not shown repeatedly.
- Local control-room dashboard: incident list, priority, transcript, Room/zone, hop count, relay latency, voice transfer completeness and playback.
- At least one localization signal: nearest fixed beacon/relay mapped to a logical zone. RSSI-based distance must be labelled approximate.
- Zone-level precursor score can be demonstrated with simulated density/SOS inputs; it is separate from per-incident triage.
- All critical behavior produces timestamped logs so the team can prove latency, hop count, delivery success and failure behavior.

1.2 Explicit non-goals for the hackathon

- No claim of Bluetooth SIG Mesh certification. The phone MVP is an application-layer BLE store-and-forward overlay implemented from Flutter using `universal\_ble` for central/peripheral BLE roles, with an optional Android platform-channel/Pigeon escape hatch only for capabilities that a plugin cannot expose reliably. Production fixed relays may use a standards-compliant Bluetooth Mesh stack.
- No live voice call or continuous audio streaming over the emergency mesh.
- No exact crowd headcount, facial recognition, identity tracking or autonomous life-critical dispatch.
- No promise of perfect background operation on every Android OEM. The Flutter MVP enters a visible active-event mode and uses `flutter\_foreground\_task` or a narrowly scoped Android foreground-service bridge when background BLE continuity is required. Background behavior must be verified on every demo phone.
- No production-grade public-key enrollment ceremony in the 48-hour demo. The demo provisioning shortcut is documented and isolated from the production security design.
- No invented fourth feature. Implement only after the product team names it.

1.3 Safety invariants

Invariant	Implementation rule
Manual SOS is never blocked	STT/triage exceptions degrade to raw SOS + audio; the send action persists before inference completes.
SOS always outranks chat/audio	Scheduler priorities are enforced centrally; Room chat cannot starve emergency metadata.

Uncertainty is visible	STT confidence, triage confidence and localization uncertainty are fields, not hidden model internals.
No live-stream assumption	Voice clips are duration-capped and store-and-forward; transfer status is explicit.
Human authority remains final	Triage/precursor outputs recommend; they do not dispatch or open gates automatically.
Graceful degradation	If MTU, audio codec, STT or beacon data is unavailable, compact SOS transport continues.

2. Engineering decisions that make the hackathon build feasible

2.1 Flutter mobile application, Android-first physical-device deployment

The mobile application is built in Flutter/Dart. Android is the first deployment target because the hackathon requires phones to perform BLE central/client and peripheral/advertiser functions, and the chosen Flutter BLE layer (`universal_ble`) exposes those roles on Android. Keep all product logic, protocol orchestration, persistence and UI in Dart. Use Flutter platform channels/Pigeon only as a controlled escape hatch for Android-specific foreground-service or OEM BLE behavior that cannot be solved cleanly in the package layer. [SRC-F01][SRC-F02][SRC-F04]

Test on physical Android phones from hour 1. Flutter hot reload accelerates UI iteration, but an emulator is not a substitute for real BLE radio behavior, microphone capture, OEM background restrictions, MTU negotiation, codec execution or native STT inference.

2.2 Do not claim full Bluetooth SIG Mesh on phones in the MVP

Bluetooth Mesh is a standardized many-to-many, multi-hop technology, but the phone MVP described here is intentionally an application-layer BLE overlay. In Flutter, `universal_ble` supplies discovery, connections, GATT and peripheral-mode primitives; MeshSetu supplies the message framing, fragmentation, deduplication, priority, expiry, acknowledgements and relay policy. This avoids overclaiming SIG Mesh certification while still letting the prototype demonstrate the core resilience hypothesis. [SRC-F04][SRC-B01][SRC-B02]

Fixed beacon/relay hardware can later use nRF52/ESP32 or another embedded platform with a standards-compliant Mesh stack. Keep application envelopes independent of the underlying bearer so the transport can evolve.

2.3 Discovery via advertising; data via GATT

- BLE advertising is used only to discover nearby MeshSetu peers and advertise a compact protocol/site fingerprint.
- Actual messages travel over a GATT connection. Each phone exposes a small GATT server and can also connect as a GATT client.
- Transport negotiates or observes the effective GATT MTU and fragments every object accordingly. `universal_ble` exposes MTU requests on supported platforms, but the application must treat the result as best-effort and never hard-code a large packet size. [SRC-F04][SRC-A03]
- If effective MTU is too small for practical voice transfer, send structured SOS/transcript and mark voice as deferred/unavailable. This is graceful degradation, not failure.

2.4 Object encryption before fragmentation

Serialize the complete application object, encrypt/authenticate it once, then fragment ciphertext into transport frames. This avoids paying an AEAD authentication tag on every tiny BLE frame. Transport frames can be dropped/reordered by untrusted relays; final object integrity is checked after reassembly.

2.5 Local-first database is the queue

The Drift database is not merely UI storage. It is the durable outbox/inbox for the Flutter app: every event has a state machine (CREATED -> READY -> RELAYING -> ACKED/EXPIRED). This makes retries, process restarts and dashboard evidence much easier to reason about. Product 'Rooms' and the Drift/SQLite database are separate concepts. [SRC-F06][SRC-F07]

2.6 Model work is an adapter, not a dependency trap

The app depends on an `OfflineSttEngine` interface and a `TriageEngine` interface. The STT developer can switch between whisper.cpp and sherpa-onnx without forcing the networking developers to rewrite UI or transport. The raw voice clip and manual SOS remain usable even when the model is not loaded.

3. End-to-end system architecture

3.1 Logical topology

Architecture sketch



Fixed BLE beacons/relays provide stable zone IDs, extra relay capacity and optional gateways.

3.2 Source-phone pipeline

1. Create a local SOS draft immediately and persist it with a UUID/event timestamp.
2. Capture typed/tap input or short PCM voice. Voice capture is capped (recommended demo default: 10 seconds).
3. Run STT locally on PCM. In parallel, encode PCM to Opus or the selected low-bitrate speech codec.
4. Run deterministic safety rules first, then optional compact classifier, producing Structured SOS + confidence/rationale.
5. Serialize event envelope (Protocol Buffers), encrypt/authenticate, fragment for current GATT MTU.
6. Place metadata/transcript object at SOS priority. Place voice object at VOICE_EVIDENCE priority.
7. Advertise availability, discover peers, connect, transfer missing frames, retry until complete/acknowledged or expired.

3.3 Relay-phone pipeline

8. Receive a frame; validate protocol version and coarse size limits.
9. Persist frame/object state before acknowledging when reliability requires it.
10. Dedupe using object ID + chunk index and suppress repeated forwarding.

11. When object completes, verify authenticated envelope, expiry and site/room scope.
12. If relay policy permits, enqueue for additional peers with hop count/TTL updated.
13. Never let voice chunks or Room chat consume all transfer slots while SOS metadata is waiting.

3.4 Gateway/control-room pipeline

14. Gateway phone receives exactly like any other peer and marks itself as a sink/bridge capability in HELLO.
15. Reassembled, verified incidents are sent over a local Wi-Fi LAN to a laptop HTTP endpoint. Internet is not required.
16. FastAPI stores current demo state and broadcasts incident updates to browser dashboards over WebSocket.
17. Operators acknowledge/dispatch in the dashboard; a signed responder update can be sent back through the gateway into the mesh as a high-priority control event.

3.5 Failure domains

Failure	Expected behavior
Internet/cellular loss	No effect on local BLE loop; dashboard stays local if gateway and laptop share LAN.
STT fails or model not loaded	Manual/voice SOS persists; transcript marked unavailable; audio + raw category sent.
Triage model uncertain	Manual SOS remains; priority falls back to rules/default; uncertainty displayed.
Voice transfer incomplete	Structured SOS remains actionable; receiver shows missing chunk count and retries until expiry.
Relay leaves range	Store-and-forward retries with other peers; TTL/expiry prevents indefinite propagation.
Beacon missing	Use logical zone chosen by user / nearest available anchor; uncertainty increases.
Gateway fails	Incidents remain locally cached and continue peer-to-peer; another node can be promoted as gateway.

4. Repository and module architecture

4.1 Recommended monorepo

```

meshsetu/
├── mobile/
│   └── lib/
│       ├── app/
│       │   ├── core/model/
│       │   ├── core/protocol/
│       │   ├── core/data/
│       │   ├── core/ble/
│       │   ├── core/security/
│       │   ├── feature/join/
│       │   ├── feature/rooms/
│       │   ├── feature/sos/
│       │   ├── feature/voice/
│       │   ├── feature/stt/
│       │   ├── feature/triage/
│       │   ├── feature/map/
│       │   ├── feature/gateway/
│       │   ├── benchmark/
│       │   └── assets/models/
│       │
│       │   # Flutter app shell, routes, themes, providers
│       │   # Dart domain models and enums
│       │   # protobuf mapping, frame codec, fragmentation
│       │   # Drift DB, repositories, durable outbox/inbox
│       │   # universal_ble adapter + peer/session orchestration
│       │   # encryption, secure key wrapper
│       │   # Mesh Code / QR join
│       │   # scoped Rooms UI + ACL policy
│       │   # SOS composition + state machine
│       │   # record, Opus encode, chunk lifecycle, playback
│       │   # OfflineSttEngine Dart interface/adapters
│       │   # rules + optional local classifier
│       │   # beacon/zone visualization
│       │   # local dashboard HTTP bridge
│       │   # on-device latency/radio/STT metrics
│       │   # packaged STT/triage models

```

```

assets/manifests/      # demo deployment manifests
android/              # Flutter runner + optional narrow native bridge
ios/                  # future; not hackathon target
pubspec.yaml
native/
  stt/                # whisper.cpp/sherpa C/C++ wrapper for Dart FFI
  opus/               # libopus wrapper for Dart FFI
protocol/             # .proto source of truth
dashboard/            # FastAPI + browser UI
ml/                   # triage training/evaluation scripts
tools/                # failure injection, metrics parser
docs/                 # runbooks, diagrams, ADRs

```

4.2 Dependency direction

Flutter feature screens depend on repositories/use-cases and Dart core packages, not directly on platform APIs. `core/ble` exposes a small Dart transport contract around `universal\_ble`; `core/data` owns Drift persistence; `feature/stt` implements the STT interface through Dart FFI or a package adapter. The STT engine must not call BLE directly, and the BLE layer must not know which ASR engine is installed. Riverpod providers bind these pieces at the application boundary. [SRC-F03][SRC-F04][SRC-F06][SRC-F08]

```

Flutter UI/features -> Riverpod use-cases/repositories -> core-data/core-protocol -> core-ble
|
+--> feature-stt (Dart FFI) ----+
+--> feature-triage -----+
+--> feature-voice (record/Opus)+

```

Native C/C++ is isolated behind Dart FFI. Android platform channels/Pigeon are an escape hatch, not the primary application architecture.

Dashboard is a separate process; it communicates only with the gateway API contract.

5. Toolchain, packages and official documentation

5.1 Baseline toolchain

Versions below are an implementation baseline checked against official release pages around 15 Aug 2026. Pin them in the repository. If a version conflict appears during setup, resolve against official compatibility guidance rather than blindly upgrading one library.

Component	Recommended baseline	Why
Flutter SDK	Pin one current stable release on all team machines	Primary mobile framework, build tool and UI runtime. Commit the same SDK expectation and pubspec.lock.
Dart SDK	Bundled with the pinned Flutter SDK	Application language, async runtime, test tooling and FFI entry point.
Android SDK / minSdk	Android-first; minSdk 29 for hackathon	Keeps the physical-device matrix manageable and supports the chosen voice path. Flutter remains the app framework.
universal_ble	2.1.1 checked 15 Aug 2026; pin tested patch	Flutter BLE central + peripheral roles, scan/connect/GATT/advertising. [SRC-F04]
flutter_riverpod	Pin tested stable	State management, async orchestration and dependency boundaries. [SRC-F08]

Drift	2.34.3	Reactive SQLite for durable outbox/inbox and audit state. [SRC-F06]
drift_flutter	0.3.1	Flutter-specific database opening/helpers. [SRC-F07]
record	7.1.1	PCM16 microphone stream for STT plus optional direct recording modes. [SRC-F05]
mobile_scanner	Pin tested stable	QR/Mesh Code scanning in Flutter. [SRC-F09]
protobuf + protoc_plugin	Pin tested stable	Compact schema-evolving objects and Dart code generation. [SRC-P01][SRC-P02]
cryptography + flutter_secure_storage	Pin tested stable	Dart AEAD plus platform-backed secret storage. [SRC-F12][SRC-F13]
flutter_foreground_task	Pin tested stable	Visible Android active-event/background execution wrapper. [SRC-F14]
tflite_flutter	0.12.1	Optional local triage classifier inference from Dart. [SRC-F11]
FastAPI	Pin current stable in venv	Local dashboard HTTP + WebSocket server. [SRC-W01]

5.2 Package/library decision matrix

Area	Primary choice	Alternative	Decision rule
BLE	universal_ble from Flutter/Dart	Small Android platform-channel/Pigeon bridge; embedded SIG Mesh for fixed relays	Start package-first. Native bridge only if a measured Android capability gap blocks the demo. [SRC-F02][SRC-F04]
UI	Flutter Material widgets	Custom design system later	Flutter owns all participant/responder mobile screens.
State / DI	flutter_riverpod	Manual constructors/providers	Use Riverpod for async session, peers, outbox and feature boundaries. [SRC-F08]

Persistence	Drift / SQLite	Raw sqflite	Durable typed outbox/inbox; do not confuse DB with product Rooms. [SRC-F06][SRC-F07]
Serialization	Protocol Buffers Dart	CBOR/custom binary	Protobuf for application objects; custom compact frame only at BLE transport layer. [SRC-P01]
Voice capture/playback	record + just_audio	Direct platform bridge only if required	Capture PCM16 once for STT + Opus; play verified reassembled audio in Flutter. [SRC-F05][SRC-F10]
Voice codec	libopus through dart:ffi	record direct Opus on supported Android	FFI gives deterministic bitrate/control while keeping product code Dart-first. [SRC-F03][SRC-O03]
STT	whisper.cpp or sherpa-onnx behind Dart FFI adapter	Maintained Flutter-compatible binding	Benchmark physical phones, then freeze one backend. [SRC-S01][SRC-S02]
Triage runtime	Rules + tflite_flutter small classifier	ONNX behind same interface	Rules remain mandatory fallback; one model runtime for hackathon. [SRC-F11]
QR scan	mobile_scanner	Manual Mesh Code entry	Package-first QR flow with manual fallback. [SRC-F09]
Gateway / dashboard	Dart `http` -> FastAPI + browser UI	Persistent WebSocket client later	HTTP is simplest for phone-to-laptop bridge; dashboard pushes browser updates by WebSocket. [SRC-F15][SRC-W01]

5.3 Flutter pubspec and build skeleton

```
# mobile/pubspec.yaml - pin exact tested versions before the demo
name: meshsetu_mobile
description: Offline-first emergency BLE relay client
publish_to: 'none'

environment:
  sdk: '>=3.10.0 <4.0.0'

dependencies:
  flutter:
    sdk: flutter
```

```

universal_ble: ^2.1.1
flutter_riverpod: <PINNED_STABLE>
drift: ^2.34.3
drift_flutter: ^0.3.1
record: ^7.1.1
just_audio: <PINNED_STABLE>
mobile_scanner: <PINNED_STABLE>
protobuf: <PINNED_STABLE>
http: <PINNED_STABLE>
cryptography: <PINNED_STABLE>
flutter_secure_storage: <PINNED_STABLE>
flutter_foreground_task: <PINNED_STABLE>
path_provider: <PINNED_STABLE>
uuid: <PINNED_STABLE>
tflite_flutter: ^0.12.1

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: <PINNED_STABLE>
  build_runner: <PINNED_STABLE>
  drift_dev: <PINNED_STABLE>
  protoc_plugin: <PINNED_STABLE>

flutter:
  uses-material-design: true
  assets:
    - assets/models/
    - assets/manifests/

# Bootstrap / code generation / device run
flutter doctor -v
flutter devices
flutter pub get
dart run build_runner build --delete-conflicting-outputs
protoc --dart_out=lib/generated -I../protocol ../protocol/meshsetu.proto
flutter run -d <DEVICE_ID>

# Quality gates and Android artifact
dart format --set-exit-if-changed lib test
flutter analyze
flutter test
flutter build apk --debug

# Native STT/Opus libraries are built by the Flutter Android toolchain/CMake
# when wired into mobile/android or through an FFI package.

```

Version placeholders are intentional: Flutter and package releases change independently. Pin a single tested `pubspec.lock` and Flutter SDK version for all three developers before feature work begins. Record the exact Flutter/Dart version in the README and do not upgrade packages during the final 12 hours unless a blocking defect requires it. [SRC-F01]

6. Core data contracts and Protocol Buffers

6.1 Application event envelope

Use a stable application schema independent of BLE. A structured SOS and a voice clip are separate objects linked by `event\_id` / `voice\_clip\_id`. This is what allows metadata to arrive even when audio is incomplete.

```

// protocol/meshsetu.proto
syntax = "proto3";
package meshsetu.v1;

// protocol/meshsetu.proto
// Generate Dart code with protoc_plugin; keep the schema platform-neutral.
// Example: protoc --dart_out=mobile/lib/generated -Iprotocol protocol/meshsetu.proto

message MeshEnvelope {
  string event_id = 1;           // UUID string in MVP; production may use bytes
  string site_id = 2;
  string room_id = 3;
  int64 created_at_ms = 4;
  int64 expires_at_ms = 5;
  uint32 hop_count = 6;
  uint32 hop_limit = 7;
  Priority priority = 8;
  PayloadType payload_type = 9;
  bytes payload = 10;           // serialized StructuredSos / RoomMessage / VoiceManifest...
  string origin_ephemeral_id = 11;
  bytes trace_id = 12;
}

```

```

enum Priority { P_UNSPECIFIED=0; P0_CRITICAL=1; P1_HIGH=2; P2_NORMAL=3; P3_BULK=4; }
enum PayloadType {
  PT_UNSPECIFIED=0; STRUCTURED_SOS=1; ROOM_MESSAGE=2; VOICE_MANIFEST=3;
  VOICE_OBJECT=4; ACK=5; RESPONDER_UPDATE=6; BEACON_OBSERVATION=7;
}

message StructuredSos {
  string incident_type = 1;
  string transcript = 2;
  float stt_confidence = 3;
  Priority triage_priority = 4;
  float triage_confidence = 5;
  repeated string hazards = 6;
  repeated string symptoms = 7;
  string location_hint = 8;
  string logical_zone = 9;
  repeated string rationale = 10;
  repeated string missing_info = 11;
  string voice_clip_id = 12;
  InputMode input_mode = 13;
}

enum InputMode { INPUT_UNSPECIFIED=0; TAP=1; TEXT=2; VOICE=3; }

message VoiceManifest {
  string voice_clip_id = 1;
  string event_id = 2;
  string codec = 3;           // "opus"
  uint32 sample_rate_hz = 4; // e.g. 16000
  uint32 channels = 5;        // 1
  uint32 duration_ms = 6;
  uint32 encoded_bytes = 7;
  bytes sha256 = 8;
}

message RoomMessage {
  string message_id = 1;
  string sender_ephemeral_id = 2;
  string text = 3;
  int64 sent_at_ms = 4;
  bool authority_signed = 5;
}

```

6.2 Dart domain model

```

final class StructuredIncident {
  const StructuredIncident({
    required this.eventId, required this.siteId, required this.roomId,
    required this.incidentType, required this.priority,
    this.transcript, this.sttConfidence, this.triageConfidence,
    this.logicalZone, required this.createdAtMs, required this.expiresAtMs,
  });

  final String eventId, siteId, roomId;
  final IncidentType incidentType;
  final PriorityBand priority;
  final String? transcript, logicalZone;
  final double? sttConfidence, triageConfidence;
  final int createdAtMs, expiresAtMs;
}

enum PriorityBand { p0Critical, p1High, p2Normal, p3Bulk }
enum IncidentType { medical, fire, crowdPressure, security, lostPerson, other }

```

6.3 Transport frame vs application envelope

Do not serialize a full protobuf envelope into every BLE write. Encrypt the envelope once, then fragment it into compact transport frames. The frame header exists only for reassembly and scheduling.

```

// 16-byte MeshSetu frame header (network byte order)
// [0] version
// [1] frameType
// [2] priority
// [3] flags
// [4..11] objectId64
// [12..13] sequenceNo (uint16)
// [14..15] chunkCount (uint16)
// [16..] fragment payload

const int frameHeaderBytes = 16;

int maxFragmentPayload(int mtu) {
  final attValueBytes = (mtu - 3).clamp(20, 65535); // conservative ATT overhead
  return (attValueBytes - frameHeaderBytes).clamp(1, 65535);
}

```

Voice policy: if `maxFragmentPayload` is too small for bounded audio within the configured expiry (for example effective MTU below a team-tested threshold), transmit only the Structured SOS and set `audio\_state=DEFERRED\_MTU`. Do not saturate the mesh trying to force audio through an unsuitable link.

7. BLE discovery, GATT transport and phone-to-phone relay

7.1 Flutter/Android permissions

```
<!-- mobile/android/app/src/main/AndroidManifest.xml -->
<uses-feature android:name="android.hardware.bluetooth_le" android:required="true" />
<uses-permission android:name="android.permission.RECORD_AUDIO" />

<!-- Android 12+ -->
<uses-permission android:name="android.permission.BLUETOOTH_SCAN"
  android:usesPermissionFlags="neverForLocation" />
<uses-permission android:name="android.permission.BLUETOOTH_ADVERTISE" />
<uses-permission android:name="android.permission.BLUETOOTH_CONNECT" />

<!-- Legacy only -->
<uses-permission android:name="android.permission.BLUETOOTH" android:maxSdkVersion="30" />
<uses-permission android:name="android.permission.BLUETOOTH_ADMIN" android:maxSdkVersion="30" />

<uses-permission android:name="android.permission.FOREGROUND_SERVICE" />
<uses-permission android:name="android.permission.FOREGROUND_SERVICE_CONNECTED_DEVICE" />
```

Flutter still ships an Android application package, so Android runtime permissions remain mandatory. Request BLE permissions through the chosen Flutter BLE layer (or a tiny permission adapter) and microphone permission through the audio flow before use. Test at least one Android 12/13 device and one recent device because permission and foreground-service behavior varies by OS release. [SRC-F04][SRC-A04]

7.2 UUID contract

```
abstract final class MeshGatt {
  static const service = '6e400001-b5a3-f393-e0a9-e50e24dcca9e';
  static const rx      = '6e400002-b5a3-f393-e0a9-e50e24dcca9e'; // peer writes frames
  static const tx      = '6e400003-b5a3-f393-e0a9-e50e24dcca9e'; // notifications/frames
}
```

These UUIDs are example development UUIDs. Generate and freeze project-specific UUIDs before publishing the repo. Never reuse vendor UUIDs from unrelated products.

7.3 Advertise only discovery metadata

```
Future<void> startMeshPeripheral() async {
  final caps = await UniversalBlePeripheral.getCapabilities();
  if (!caps.supportsPeripheralMode) {
    throw StateError('BLE peripheral mode unavailable on this device');
  }

  await UniversalBlePeripheral.addService(
    BlePeripheralService(
      uuid: MeshGatt.service,
      primary: true,
      characteristics: [
        BlePeripheralCharacteristic(
          uuid: MeshGatt.rx,
          properties: [CharacteristicProperty.write],
          permissions: [PeripheralAttributePermission.writeable],
        ),
        BlePeripheralCharacteristic(
          uuid: MeshGatt.tx,
          properties: [CharacteristicProperty.notify],
          permissions: [PeripheralAttributePermission.readable],
        ),
      ],
    ),
  );

  await UniversalBlePeripheral.startAdvertising(
    services: const [MeshGatt.service],
    localName: 'MeshSetu',
  );
}

// Site/event identity is verified in the HELLO/application layer.
// Do not treat the advertised name or service UUID as authentication.
```

```
Future<void> stopMeshPeripheral() async {
  await UniversalBlePeripheral.stopAdvertising();
  await UniversalBlePeripheral.clearServices();
}
```

7.4 Scan peers with a time-bounded Flutter window

```
Future<List<BleDevice>> scanPeers({
  Duration window = const Duration(seconds: 4),
}) async {
  final found = <String, BleDevice>{};
  final sub = UniversalBle.scanStream.listen((device) {
    found[device.deviceId] = device;
  });

  await UniversalBle.startScan(
    scanFilter: ScanFilter(withServices: const [MeshGatt.service]),
    platformConfig: const PlatformConfig(
      android: AndroidOptions(scanMode: AndroidScanMode.lowLatency),
    ),
  );
  await Future<void>.delayed(window);
  await UniversalBle.stopScan();
  await sub.cancel();
  return found.values.toList(growable: false);
}
```

Scanning must be duty-cycled instead of left permanently in high-power mode. The Flutter layer owns the scan/connect/idle policy, while Android remains the underlying radio platform. `universal\_ble` also documents Android background scanning through foreground-task services after permissions are already granted. Measure peer-discovery latency and battery cost on the actual demo devices. [SRC-F04]

7.5 Flutter BLE peripheral: receive frames

```
void installPeripheralWriteHandler(StreamController<PeerFrame> frameInbox) {
  UniversalBlePeripheral.setWriteRequestHandlers(
    (deviceId, characteristicId, offset, value) {
      if (characteristicId == MeshGatt.rx && offset == 0) {
        frameInbox.add(PeerFrame(peerId: deviceId, bytes: Uint8List.fromList(value)));
      }
    },
    return PeripheralWriteRequestResult();
  );
}

Future<void> notifyPeer(String deviceId, Uint8List frame) async {
  await UniversalBlePeripheral.updateCharacteristicValue(
    characteristicId: MeshGatt.tx,
    value: frame,
    deviceId: deviceId,
  );
}
```

7.6 Flutter BLE central connection and MTU

```
Future<PeerSession> connectPeer(BleDevice device) async {
  await device.connect();
  await device.discoverServices();

  final negotiatedMtu = await device.requestMtu(247);
  final rx = await device.getCharacteristic(MeshGatt.service, MeshGatt.rx);
  final tx = await device.getCharacteristic(MeshGatt.service, MeshGatt.tx);

  await tx.notifications.subscribe();
  return PeerSession(device: device, rx: rx, tx: tx, mtu: negotiatedMtu);
}
```

Treat MTU requests as best-effort. The actual transport frame size must be derived from the effective MTU reported by the BLE package/device combination, and every path must still work at conservative values. Android platform behavior can change across releases, so never make the 517-byte case a protocol requirement. [SRC-F04][SRC-A03]

7.7 Flutter write helper

```
Future<void> writeFrame(PeerSession peer, Uint8List frame) async {
  if (frame.length > maxFragmentPayload(peer.mtu)) {
    throw ArgumentError('frame exceeds negotiated payload budget');
  }
  await peer.rx.write(frame, withResponse: false);
}
```

8. Store-and-forward mesh protocol

8.1 Node identity and peer HELLO

Use a rotating ephemeral node ID per event/session, not a stable advertising identifier. HELLO contains only what peers need: protocol version, site fingerprint, capabilities and a short queue summary.

```
final class Hello {
    const Hello({
        this.protocolVersion = 1,
        required this.siteFingerprint, required this.ephemeralNodeId,
        required this.capabilities, required this.maxObjectBytes,
    });
    final int protocolVersion, siteFingerprint, ephemeralNodeId;
    final int capabilities; // RELAY | GATEWAY | BEACON | VOICE | STT
    final int maxObjectBytes;
}
```

8.2 Priority scheduler

```
enum TrafficClass { controlAck, sosStructured, authorityControl, voiceEvidence, roomMessage, telemetry }

extension TrafficRank on TrafficClass {
    int get rank => switch (this) {
        TrafficClass.controlAck => 0,
        TrafficClass.sosStructured => 1,
        TrafficClass.authorityControl => 2,
        TrafficClass.voiceEvidence => 3,
        TrafficClass.roomMessage => 4,
        TrafficClass.telemetry => 5,
    };
}

final class OutboundObject {
    const OutboundObject({required this.objectId, required this.trafficClass, required this.createdAtMs, required
this.expiresAtMs, required this.bytes});
    final int objectId, createdAtMs, expiresAtMs;
    final TrafficClass trafficClass;
    final Uint8List bytes;
}

final queue = <OutboundObject>[];
void enqueue(OutboundObject object) {
    queue.add(object);
    queue.sort((a, b) {
        final p = a.trafficClass.rank.compareTo(b.trafficClass.rank);
        return p != 0 ? p : a.createdAtMs.compareTo(b.createdAtMs);
    });
}
```

8.3 Fragmentation

```
final class MeshFrame {
    const MeshFrame({
        this.version = 1, required this.type, required this.priority, required this.flags,
        required this.objectId, required this.sequence, required this.count, required this.payload,
    });
    final int version, type, priority, flags, objectId, sequence, count;
    final Uint8List payload;
}

List<MeshFrame> fragment({required int objectId, required int priority, required Uint8List encrypted, required int
mtu}) {
    final n = maxFragmentPayload(mtu);
    final frames = <MeshFrame>[];
    for (var offset = 0, seq = 0; offset < encrypted.length; offset += n, seq++) {
        final end = (offset + n).clamp(0, encrypted.length);
        frames.add(MeshFrame(type: 1, priority: priority, flags: 0, objectId: objectId, sequence: seq, count:
(encrypted.length / n).ceil(), payload: Uint8List.sublistView(encrypted, offset, end)));
    }
    return frames;
}
```

8.4 Reassembly and integrity

```
final class ReassemblyBuffer {
    ReassemblyBuffer(this.count, this.createdAtMs) : _parts = List<Uint8List?>.filled(count, null);
    final int count, createdAtMs;
    final List<Uint8List?> _parts;
    int received = 0;

    bool add(int seq, Uint8List bytes) {
```

```

    if (seq < 0 || seq >= count || _parts[seq] != null) return false;
    _parts[seq] = bytes; received++; return true;
  }
  bool get complete => received == count;
  Uint8List join() { final b = BytesBuilder(copy: false); for (final p in _parts) { if (p == null) throw
  StateError('incomplete'); b.add(p); } return b.takeBytes(); }
}

```

8.5 Dedupe and anti-replay

```

final class RecentObjectCache {
  RecentObjectCache({this.maxEntries = 4096});
  final int maxEntries;
  final _seen = <int, int>{};

  bool firstSeen(int objectId, int nowMs) {
    if (_seen.containsKey(objectId)) return false;
    _seen[objectId] = nowMs;
    if (_seen.length > maxEntries) _seen.remove(_seen.keys.first);
    return true;
  }
}

```

8.6 Relay algorithm

```

Future<void> onCompleteObject(Uint8List ciphertext, TransportMeta transport) async {
  final envelope = await crypto.decryptAndVerify(ciphertext);
  if (envelope == null) { metrics.invalidObject(); return; }
  if (clock.nowMs() > envelope.expiresAtMs) { metrics.expired(); return; }
  if (!recent.firstSeen(envelope.objectId, clock.nowMs())) { metrics.duplicate(); return; }

  await inbox.persist(envelope);
  uiEvents.add(envelope);

  if (policy.shouldRelay(envelope, transport)) {
    await scheduler.enqueueForRelay(envelope, excludePeer: transport.peerId);
  }
}

bool shouldRelay(Envelope e, TransportMeta t) {
  if (e.hopsRemaining <= 0 || DateTime.now().millisecondsSinceEpoch > e.expiresAtMs) return false;
  if (e.trafficClass == TrafficClass.telemetry && scheduler.sosBacklog > 0) return false;
  return true;
}

```

Production note: mutating hop count inside an encrypted/authenticated envelope requires decrypt/re-encrypt by trusted site members. Another design is an immutable signed origin envelope wrapped by mutable relay metadata. The MVP can use shared site encryption; production should separate origin authenticity from relay routing metadata.

8.7 Reliability strategy

- ACK complete objects, not every frame, unless tests prove frame-level ACKs are necessary.
- Receiver can send a compact missing-chunk bitmap/NACK for voice objects.
- Use bounded retries with jitter; do not create synchronized retry storms.
- Persist the outbox before transmit. Remove/age out only after ACK or expiry.
- Cap object size, chunk count, retry count and per-peer transfer time to resist accidental or malicious exhaustion.
- Prefer opportunistic replication to 1-2 good peers rather than flooding every byte to every discovered phone immediately; benchmark fan-out.

9. Mesh Code / QR provisioning

9.1 What the Mesh Code is

The Mesh Code is a human UX/bootstrap identifier for the event/site namespace. A six-character code is not enough entropy to be a network root key. For a fully offline hackathon, support two paths: QR contains a signed demo manifest; typed code selects a manifest that was bundled/preloaded on the app.

9.2 Event manifest

```
final class EventManifest {
  const EventManifest({required this.siteId, required this.siteName, required this.meshCode, required
this.validFromMs, required this.validUntilMs, required this.rooms, required this.gatewayHint});
  final String siteId, siteName, meshCode, gatewayHint;
  final int validFromMs, validUntilMs;
  final List<RoomManifest> rooms;
}

final class RoomManifest {
  const RoomManifest({required this.roomId, required this.name, required this.role, required this.ttlSeconds});
  final String roomId, name, role;
  final int ttlSeconds;
}

enum RoomAccess { public, volunteer, medical, responder, authority }

// Product Rooms are scoped communication channels; this enum is authorization metadata,
// not a database type.
```

9.3 QR scan

Use the Flutter `mobile\_scanner` package for QR/Mesh Code capture. Keep manual code entry as a fallback. The QR contains only a signed/validated event manifest or compact join token; it should never contain long-lived authority secrets. [SRC-F09]

```
MobileScanner(
  controller: MobileScannerController(formats: const [BarcodeFormat.qrCode]),
  onDetect: (capture) {
    if (capture.barcodes.isEmpty) return;
    final raw = capture.barcodes.first.rawValue;
    if (raw != null) ref.read(joinControllerProvider.notifier).acceptQr(raw);
  },
)

Future<JoinResult> parseAndValidateJoinCode(String raw) async {
  final manifest = EventManifestCodec.decode(raw);
  if (!manifest.signatureValid) return JoinResult.invalid('signature');
  if (clock.nowMs() > manifest.validUntilMs) return JoinResult.invalid('expired');
  await repository.activateManifest(manifest);
  return JoinResult.ok(manifest);
}
```

9.4 Validation checklist

- Verify manifest signature before activation.
- Reject expired/not-yet-valid manifests.
- Show site/event name and expiry to user before joining.
- Do not leak restricted Room keys to public attendees.
- Demo shortcut ('demoSiteKeyB64' in QR) must be compiled only into a hackathon flavor and visibly documented as non-production.

10. Rooms and operational message routing

10.1 Room semantics

Room	Audience	Allowed content	Transport priority
Public Alerts	All participants	Authority notices + limited user acknowledgements	Authority control > normal chat
Zone: Gate-B	People/volunteers in a logical zone	Local guidance, queue updates	Normal
Medical	Authorized medical/volunteer roles	Medical coordination + SOS references	High but below SOS metadata

Responders	Authority/responder roles	Dispatch/operational updates	Authority control
Family Reunification	Optional public/assisted role	Lost/found person coordination without sensitive over-sharing	Normal

10.2 Room ACL and transport policy

```
final class RoomPolicy {
  const RoomPolicy({required this.roomId, required this.sendRoles, required this.readRoles, required
this.trafficClass, required this.ttlSeconds});
  final String roomId;
  final Set<String> sendRoles, readRoles;
  final TrafficClass trafficClass;
  final int ttlSeconds;
}

bool canSend(RoomPolicy room, Set<String> userRoles) => room.sendRoles.any(userRoles.contains);
bool canRead(RoomPolicy room, Set<String> userRoles) => room.readRoles.any(userRoles.contains);
```

Room membership controls visibility and forwarding. Restricted Rooms should use separate room keys in production. A public participant should not be able to decrypt responder traffic simply because they know the event Mesh Code.

10.3 SOS transcends Room chat

An SOS may carry a `room\_id` for operator context, but relay eligibility must not depend on ordinary Room subscription. Emergency metadata is an event-level safety object. This prevents a phone outside “Medical” from refusing to relay a medical SOS.

11. Voice capture, compression, packetization and playback

11.1 Exact feature behavior

Voice is an input and evidence mode, not a voice-call feature. Sender records a short clip; the app produces two outputs in parallel: (A) PCM for on-device STT and (B) an Opus-encoded bounded voice object. The structured/transcribed SOS is sent at emergency priority before the voice object. Bluetooth Mesh itself is not an audio-streaming technology; the design deliberately uses store-and-forward short messages. [SRC-B01][SRC-O01]

11.2 Audio capture format

The Flutter voice path captures 16 kHz mono PCM16 with the `record` package so the same raw audio can feed offline STT and the Opus encoder. The app remains Flutter/Dart; platform microphone APIs are reached through the package. Keep recording duration capped (for example 5-10 seconds) and make the user explicitly submit a clip. [SRC-F05]

```
final class PcmRecorder {
  PcmRecorder(this._recorder);
  final AudioRecorder _recorder;

  Future<Uint8List> recordMax({Duration max = const Duration(seconds: 10)}) async {
    if (!await _recorder.hasPermission()) throw StateError('microphone permission denied');
    final stream = await _recorder.startStream(const RecordConfig(
      encoder: AudioEncoder.pcm16bits, sampleRate: 16000, numChannels: 1,
      autoGain: true, noiseSuppress: true,
    ));
    final bytes = BytesBuilder(copy: false);
    final sub = stream.listen(bytes.add);

    await Future<void>.delayed(max);
    await _recorder.stop();
    await sub.cancel();
    return bytes.takeBytes();
  }
}
```

```
// Production UI should stop immediately when the user releases/taps stop.
// The duration cap is a safety/net-load ceiling, not a forced 10 s recording.
// Store the PCM only transiently unless debugging/consent policy explicitly allows it.
```

11.3 Opus design

Opus is designed for speech and audio across a wide bitrate range. For the demo, benchmark approximately 8-12 kbit/s mono speech; do not hard-code a bitrate claim until listening tests and transfer tests succeed. A 10-second clip at 8 kbit/s is about 10 KB before container/protocol overhead, which is small enough to test as a bounded object but still large relative to a normal BLE message. [SRC-O01][SRC-O02]

Option A - PCM in Flutter + libopus through Dart FFI (recommended)

```
abstract interface class VoiceEncoder {
  Future<Uint8List> encodeOpus(Uint8List pcm16le, {int sampleRateHz = 16000, int channels = 1, int bitrate = 8000});
}

final class LibOpusVoiceEncoder implements VoiceEncoder {
  LibOpusVoiceEncoder(this._native);
  final MeshSetuOpusFfi _native; // tiny C ABI wrapper loaded with dart:ffi

  @override
  Future<Uint8List> encodeOpus(Uint8List pcm16le, {int sampleRateHz = 16000, int channels = 1, int bitrate = 8000})
=>
  _native.encode(pcm16le, sampleRateHz: sampleRateHz, channels: channels, bitrate: bitrate);
}
```

Why this is the preferred Flutter path: one PCM capture can be consumed by STT and, in parallel, encoded to Opus. `record` can also write Opus directly on supported Android versions, but direct encoded recording does not give the STT engine the same PCM stream as cleanly. Therefore the product pipeline is PCM once -> local STT + local Opus encode -> structured SOS first -> compressed audio chunks second. [SRC-F05][SRC-O01][SRC-O02]

Option B - direct Opus recording for voice-only/fallback path

```
final recorder = AudioRecorder();
await recorder.start(
  const RecordConfig(encoder: AudioEncoder.opus, sampleRate: 16000, numChannels: 1, bitRate: 8000),
  path: opusPath,
);
// Later: final path = await recorder.stop();
// Use only after verifying codec support on every demo Android phone.
// native/opus/meshsetu_opus.h - deliberately small C ABI for Dart FFI
int meshsetu_opus_encode(
  const int16_t* pcm, int sample_count, int sample_rate, int channels,
  int bitrate, uint8_t* out, int out_capacity
);
// Flutter/Dart owns buffers, lifecycle and errors; libopus owns codec internals.
```

Keep the FFI surface minimal and deterministic: init/encode/free or a single encode call for short clips. Package the native library per Android ABI used in the demo. Do not pass Dart object pointers into long-running native threads. The codec is evidence transport, not a live audio stream. [SRC-F03][SRC-O03]

11.4 Voice object transfer

```
Future<void> enqueueVoiceEvidence(String eventId, Uint8List opusBytes, int durationMs) async {
  final clipId = const Uuid().v4();
  final manifest = VoiceManifest(voiceClipId: clipId, eventId: eventId, codec: 'opus', durationMs: durationMs,
byteLength: opusBytes.length, sha256: await sha256Bytes(opusBytes));
  await outbox.enqueue(manifest, TrafficClass.voiceEvidence);
  await outbox.enqueueBlob(clipId, opusBytes, TrafficClass.voiceEvidence);
}
```

11.5 Receiver playback

After all chunks arrive, verify hash/integrity, persist the Opus file, then play it in Flutter with `just\_audio` on supported devices. The receiver must visibly show incomplete/missing chunks instead of silently playing corrupted evidence. Dashboard playback can remain a gateway-side concern for the hackathon. [SRC-F10]

11.6 Voice acceptance tests

Metric	Hackathon test
--------	----------------

Capture success	10/10 clips recorded on each demo device with internet off.
STT latency	Measure recording-end → transcript-ready; report median and worst of 10 clips.
Encoded size	Record encoded bytes, duration and bitrate configuration.
Mesh transfer	Record chunks sent/received/retried and reassembly time across 1-hop and 2-hop.
Priority isolation	While voice transfers, inject a new SOS and prove structured SOS overtakes remaining audio chunks.
Corruption	Drop/alter one chunk; verify reassembly remains incomplete or integrity check fails, not silent bad playback.

12. Offline Speech-to-Text architecture and inference plan

12.1 Interface contract

```
final class SttResult {
  const SttResult({required this.text, required this.confidence, required this.inferenceMs, required this.modelId});
  final String text, modelId;
  final double confidence;
  final int inferenceMs;
}

abstract interface class OfflineSttEngine {
  Future<void> warmUp();
  Future<SttResult> transcribe(Uint8List pcm16le, {int sampleRateHz = 16000});
  Future<void> close();
}
```

Networking and UI code call only this interface. This lets the STT developer benchmark and swap engines independently.

12.2 Candidate A: whisper.cpp

whisper.cpp supports Android and integer quantization, making it a practical local multilingual ASR candidate. The team should start with a quantized tiny or base multilingual model and measure memory, real-time factor and accuracy on the actual demo phones. [SRC-S01]

```
// Flutter/Dart adapter over a small native C API exposed through dart:ffi
final class WhisperCppStt implements OfflineSttEngine {
  WhisperCppStt(this.modelPath, this._ffi);
  final String modelPath;
  final MeshSetuSttFfi _ffi;
  int? _context;

  @override
  Future<void> warmUp() async { _context = await _ffi.init(modelPath); }

  @override
  Future<SttResult> transcribe(Uint8List pcm16le, {int sampleRateHz = 16000}) async {
    final sw = Stopwatch()..start();
    final text = await _ffi.transcribe(_context!, pcm16le, sampleRateHz);
    return SttResult(text: text, confidence: 0.0, inferenceMs: sw.elapsedMilliseconds, modelId:
'whisper.cpp:<PINNED_MODEL>');
  }

  @override
  Future<void> close() async { if (_context != null) await _ffi.free(_context!); }
}

# native/stt/CMakeLists.txt sketch
cmake_minimum_required(VERSION 3.22.1)
```

```

project(meshsetu_stt)
add_subdirectory(whisper.cpp)
add_library(meshsetu_stt SHARED stt_c_api.cpp)
target_link_libraries(meshsetu_stt PRIVATE whisper log)

# stt_c_api.cpp exports a small C ABI: init / transcribe / free.
# Dart calls it via dart:ffi; no JNI is required for the primary path.

```

12.3 Candidate B: sherpa-onnx

sherpa-onnx is also designed for offline speech/audio processing. If its chosen model performs better on the target phones, expose it behind the same Dart `OfflineSttEngine` contract through its native C/C++ API or a maintained Flutter-compatible binding. The Flutter product code must not change when the STT backend changes. [SRC-S02][SRC-F03]

The STT developer should not choose a model from name recognition. Choose it only after the benchmark matrix below. For Hindi/English or code-switched Indian use, confirm actual model language coverage and test your own noisy samples.

12.4 STT model decision gate - first 6 hours

Criterion	whisper.cpp candidate	sherpa-onnx candidate	Pass condition
Offline	Yes by architecture	Yes by architecture	Airplane mode; no network permission required for inference.
Language	Use multilingual model	Depends on selected model	Required demo languages recognized on real samples.
Model load	Measure	Measure	No OOM on weakest demo device.
Latency	Measure RTF / end latency	Measure RTF / end latency	10 s clip returns in team-defined acceptable time.
Accuracy	Measure keyword/WER proxy	Measure keyword/WER proxy	Critical emergency terms preserved reliably enough for demo.
Integration	Dart FFI to small whisper.cpp C API	Dart FFI/native sherpa binding	One stable `OfflineSttEngine` implementation callable from Flutter by hour 12.

12.5 Benchmark harness

```

final class SttBenchmarkSample {
  const SttBenchmarkSample({required this.clipId, required this.language, required this.expectedKeywords, required this.transcript, required this.modelLoadMs, required this.inferenceMs, required this.peakMemoryMb});
  final String clipId, language, transcript;
  final List<String> expectedKeywords;
  final int modelLoadMs, inferenceMs;
  final double peakMemoryMb;
}

double keywordRecall(List<String> expected, String transcript) {
  if (expected.isEmpty) return 1.0;
  final t = transcript.toLowerCase();
  final hit = expected.where((k) => t.contains(k.toLowerCase())).length;
  return hit / expected.length;
}

```

```
Future<SttResult> benchmarkOne(OfflineSttEngine engine, Uint8List pcm) async {
  final result = await engine.transcribe(pcm);
  metrics.add('stt_inference_ms', result.inferenceMs);
  return result;
}
```

12.6 Data for STT testing

- Record 20-30 consented synthetic emergency phrases spoken by the team, not real victim data.
- Include quiet room, crowd-noise playback, fast speech, different accents, Hindi/English/code-switching if that is the demo goal.
- Label critical keywords separately from exact wording: “fire”, “unconscious”, “cannot breathe”, “Gate B”, “child”, “bleeding”, “pushing”.
- Keep test clips outside app analytics and delete after benchmark unless consent explicitly allows retention.

12.7 STT confidence policy

Many ASR engines do not expose a calibrated single “confidence” directly. Do not fabricate one. If the selected engine offers token/log-probability information, map it to a clearly documented heuristic. Otherwise set `stt\_confidence=null` and show “confidence unavailable”; use transcript + original audio as evidence.

12.8 Optimization sequence

18. Get one model working on one physical phone before any quantization work.
19. Profile model load time, inference time, memory and thermals.
20. Try quantized model; compare critical-keyword retention, not just WER.
21. Limit threads to avoid starving BLE/UI. Start with 2-4 CPU threads and benchmark.
22. Warm the model when user enters active event mode if memory permits.
23. Use device acceleration (NNAPI/QNN/LiteRT NPU) only after a CPU baseline is stable; acceleration support varies by operators/model. [SRC-M01][SRC-M02]

13. On-device AI triage and structured SOS

13.1 Design principle: hybrid, conservative, explainable

For a safety-critical hackathon, the fastest credible design is deterministic critical-phrase rules + a small local text classifier. Do not use a generative LLM as the only triage engine. The model proposes structure/priority; rules can force escalation for a narrow list of safety-critical phrases; human operators remain final.

13.2 Structured SOS recipe

```
final class TriageOutput {
  const TriageOutput({required this.priority, required this.incidentType, required this.confidence, required
this.rationale, required this.modelId});
  final PriorityBand priority;
  final IncidentType incidentType;
  final double confidence;
  final List<String> rationale;
  final String modelId;
}
```

13.3 Deterministic safety rules

```
final class SafetyRules {
  static final critical = <RegExp>[
    RegExp(r'not breathing|unconscious|cannot breathe|can.t breathe', caseSensitive: false),
    RegExp(r'fire|smoke|crush|stampede|trapped', caseSensitive: false),
  ];

  TriageOutput? evaluate(String text) {
    for (final rule in critical) {
      if (rule.hasMatch(text)) {
        return const TriageOutput(priority: PriorityBand.p0Critical, incidentType: IncidentType.other, confidence:
1.0, rationale: ['critical safety rule matched'], modelId: 'rules:v1');
      }
    }
  }
}
```

```

    return null;
  }
}

```

13.4 Small classifier training pipeline (optional but recommended)

Train on a laptop; infer on-device. Keep the model small and the input contract simple: tokenizer maps a fixed vocabulary to integer IDs; model predicts incident type / base priority. Export vocabulary alongside the model. The rules engine still owns hard safety overrides.

```

# ml/triage-training/train.py (reference baseline)
import json, numpy as np, tensorflow as tf
from tensorflow import keras

MAX_LEN = 48
VOCAB = json.load(open("vocab.json"))
NUM_TYPES = 6

inp = keras.Input(shape=(MAX_LEN,), dtype="int32", name="token_ids")
x = keras.layers.Embedding(len(VOCAB), 48, mask_zero=True)(inp)
x = keras.layers.GlobalAveragePooling1D()(x)
x = keras.layers.Dense(48, activation="relu")(x)
out = keras.layers.Dense(NUM_TYPES, activation="softmax", name="incident_type")(x)
model = keras.Model(inp, out)
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])
model.fit(train_ids, train_labels, validation_data=(val_ids, val_labels), epochs=12, batch_size=32)

converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
open("incident_classifier.tflite", "wb").write(converter.convert())

```

13.5 On-device inference contract

```

abstract interface class TriageClassifier {
  Future<TriageOutput> predict(String transcript);
}

final class TriageEngineImpl {
  const TriageEngineImpl(this.rules, this.classifier);
  final SafetyRules rules;
  final TriageClassifier classifier;

  Future<TriageOutput> triage(String transcript) async {
    final forced = rules.evaluate(transcript);
    if (forced != null) return forced;
    try { return await classifier.predict(transcript); }
    catch (_) { return const TriageOutput(priority: PriorityBand.p1High, incidentType: IncidentType.other,
confidence: 0.0, rationale: ['model unavailable; conservative fallback'], modelId: 'fallback'); }
  }
}

```

13.6 LiteRT vs ONNX Runtime

If the classifier is exported as TFLite/LiteRT, call it from Dart using `tflite\_flutter` so the mobile inference layer stays inside the Flutter architecture. If the team chooses ONNX instead, place that runtime behind the same `TriageClassifier` interface. Do not ship two model runtimes during the hackathon unless both are genuinely required. [SRC-F11][SRC-M02]

13.7 Priority mapping

Band	Meaning	Examples	Action
P0 Critical	Immediate life/safety threat	Not breathing, unconscious, active fire, severe crush indicators	Relay first; control room visually/audibly flags; human dispatch.
P1 High	Urgent but not proven immediately life-threatening	Injury, escalating crowd pressure, trapped person	High relay priority; operator review.
P2 Normal	Needs assistance	Lost person, non-urgent help request	Standard queue.

P3 Bulk	Non-emergency evidence/telemetry	Voice chunks, routine Room messages	Never block SOS.
---------	----------------------------------	-------------------------------------	------------------

13.8 Model evaluation

For the hackathon, report confusion matrix / per-class recall on a small held-out synthetic set, but label it a prototype benchmark. Safety acceptance should emphasize recall on P0/P1 examples and false escalation burden. Do not claim clinical or emergency-dispatch validation.

14. Localization, density and zone precursor scoring

14.1 Beacon observations

```
final class BeaconObservation {
    const BeaconObservation({required this.beaconId, required this.zone, required this.rssi, required this.seenAtMs});
    final String beaconId, zone;
    final int rssi, seenAtMs;
}

double? rssiDistanceMeters(int rssi, int measuredPowerAt1m, {double n = 2.2}) {
    if (rssi == 0) return null;
    return pow(10, (measuredPowerAt1m - rssi) / (10 * n)).toDouble();
}
```

14.2 Zone-first localization

The hackathon should prioritize reliable logical-zone localization over fake sub-meter precision. Assign each fixed beacon a known zone. If multiple anchors are visible, use strongest/smoothed RSSI and report a confidence class. RSSI is noisy around bodies and reflective structures; the UI should show “Zone B / approximate” rather than a precise dot unless calibrated.

```
final class ZoneEstimate {
    const ZoneEstimate({required this.zone, required this.confidence, required this.basis});
    final String zone, basis;
    final double confidence;
}

ZoneEstimate estimateZone(List<BeaconObservation> obs) {
    if (obs.isEmpty) return const ZoneEstimate(zone: 'unknown', confidence: 0, basis: 'no beacon');
    final sorted = [...obs]..sort((a, b) => b.rssi.compareTo(a.rssi));
    final best = sorted.first;
    return ZoneEstimate(zone: best.zone, confidence: 0.6, basis: 'strongest beacon RSSI');
}
```

14.3 Density estimate - demo method

Density is a zone-level operational estimate from ephemeral observations, not an exact human count. In the hackathon, use either a controlled number of participating devices or a simulation feed, and expose the calibration assumption.

```
double estimateDensity({required int distinctEphemeralDevices, required double calibratedCoverageAreaM2, required double participationFactor}) {
    if (calibratedCoverageAreaM2 <= 0 || participationFactor <= 0) return 0;
    return distinctEphemeralDevices / participationFactor / calibratedCoverageAreaM2;
}
```

14.4 Precursor score

```
final class PrecursorInput {
    const PrecursorInput({required this.densityTrend, required this.sosCluster, required this.motionAnomaly, required this.persistence});
    final double densityTrend, sosCluster, motionAnomaly, persistence;
}

final class ScoreWeights {
    const ScoreWeights({this.d = .35, this.s = .35, this.m = .15, this.p = .15});
    final double d, s, m, p;
}

double precursorScore(PrecursorInput x, ScoreWeights w) =>
    w.d * x.densityTrend + w.s * x.sosCluster + w.m * x.motionAnomaly + w.p * x.persistence;
```

Keep this score separate from incident triage in code, data model and UI. It answers “is this zone becoming risky?” rather than “how urgent is this person’s SOS?”

15. Gateway and local control-room dashboard

15.1 Gateway API

Simplest hackathon bridge: gateway phone and laptop join the same local Wi-Fi hotspot. The phone POSTs verified incident JSON to the laptop. No internet access is required. The laptop broadcasts updates to browsers via WebSocket.

```
// Flutter gateway: send completed signed event to a laptop dashboard on local LAN
Future<void> postToDashboard(Uri baseUrl, Map<String, Object?> event) async {
  final uri = baseUrl.resolve('/api/events');
  final response = await http.post(
    uri,
    headers: const {'content-type': 'application/json'},
    body: jsonEncode(event),
  ).timeout(const Duration(seconds: 3));
  if (response.statusCode < 200 || response.statusCode >= 300) {
    throw StateError('dashboard HTTP ${response.statusCode}');
  }
}
```

15.2 FastAPI server

```
# dashboard/main.py
from fastapi import FastAPI, WebSocket, WebSocketDisconnect, Header, HTTPException
from pydantic import BaseModel
from typing import Optional

app = FastAPI(title="MeshSetu Local Control Room")
clients: set[WebSocket] = set()
latest: dict[str, dict] = {}
DEMO_KEY = "change-me"

class Event(BaseModel):
    event_id: str
    priority: str
    incident_type: str
    transcript: Optional[str] = None
    zone: Optional[str] = None
    room: Optional[str] = None
    hops: int = 0
    relay_latency_ms: Optional[int] = None
    voice_clip_id: Optional[str] = None
    audio_state: Optional[str] = None

@app.post("/api/events")
async def ingest(event: Event, x_meshsetu_demo_key: str = Header(default="")):
    if x_meshsetu_demo_key != DEMO_KEY:
        raise HTTPException(401, "bad demo key")
    latest[event.event_id] = event.model_dump()
    dead = []
    for ws in clients:
        try: await ws.send_json({"type": "event", "data": latest[event.event_id]})
        except Exception: dead.append(ws)
    for ws in dead: clients.discard(ws)
    return {"ok": True}

@app.get("/api/events")
def list_events():
    return list(latest.values())

@app.websocket("/ws")
async def ws_endpoint(ws: WebSocket):
    await ws.accept(); clients.add(ws)
    await ws.send_json({"type": "snapshot", "data": list(latest.values())})
    try:
        while True: await ws.receive_text()
    except WebSocketDisconnect:
        clients.discard(ws)
```

FastAPI supports WebSocket endpoints directly; this is enough for a local demo dashboard without introducing a message broker. [SRC-W01]

15.3 Minimal browser client

```
<!-- dashboard/static/index.html -->
<!doctype html><meta charset="utf-8"><title>MeshSetu Control Room</title>
<h1>MeshSetu Local Control Room</h1><div id="events"></div>
```



```

<script>
const root = document.getElementById('events');
const state = new Map();
function render(){
  root.innerHTML = [...state.values()].sort((a,b)=>a.priority.localeCompare(b.priority)).map(e => `
    <article><h3>${e.priority} · ${e.incident_type} · ${e.zone||'unknown zone'}</h3>
    <p>${e.transcript||'No transcript'}</p>
    <small>${e.hops} hops · ${e.relay_latency_ms??'?' } ms · audio
  ${e.audio_state||'n/a'}</small></article>` ).join('');
}
const ws = new WebSocket(`ws://${location.host}/ws`);
ws.onmessage = ev => { const m=JSON.parse(ev.data); (m.type==='snapshot'?m.data:
[m.data]).forEach(e=>state.set(e.event_id,e)); render(); };
</script>

```

15.4 Dashboard fields

- Event ID, priority, incident type, received timestamp.
- Transcript + STT status; button to play reassembled original voice evidence when complete.
- Room, logical zone, localization uncertainty/anchor source.
- Triage rationale and confidence; separate zone precursor score and evidence.
- Hop count, origin-to-gateway relay latency, duplicate/retry count.
- Operator ACK / dispatch state; responder update back into mesh (stretch goal).

16. Security, privacy and key management

16.1 Threat model

Threat	MVP mitigation	Production hardening
Fake SOS injection	Site-scoped shared demo auth + validation + rate limits	Per-device credentials, signed enrollment, authority trust chain, abuse controls.
Packet tampering	AEAD object integrity; hash voice bytes	Origin signatures + separate relay metadata.
Replay	UUID/object IDs + expiry + recent-object cache	Persistent replay windows / signed counters where appropriate.
Eavesdropping	Encrypt restricted payloads	Per-Room keys, forward secrecy where feasible, audited key lifecycle.
Relay exhaustion	Size caps, priorities, TTL, retries, per-peer quotas	Adaptive rate limiting, trust/reputation policy, hardened relays.
Stolen device	Short event keys + no long-term personal data	Device revocation, hardware-backed keys, MDM for responders.
Sensitive voice retention	Short retention; only authorized receiver	Explicit policy, encrypted at rest, deletion/audit, DPDP review.

16.2 Flutter secure storage and Android Keystore-backed secrets

Use `flutter\_secure\_storage` for small site/room wrapping secrets and device credentials. On Android, the plugin delegates to platform-specific secure storage and cryptography; application code must never hard-code event

keys in Dart source. Use the Dart `cryptography` package for message-level AEAD so protocol code remains testable and platform-neutral. [SRC-F12][SRC-F13]

```
const secureStorage = FlutterSecureStorage();

Future<void> storeSiteKey(String siteId, String base64Key) =>
  secureStorage.write(key: 'meshsetu.site.$siteId', value: base64Key);

Future<String?> loadSiteKey(String siteId) =>
  secureStorage.read(key: 'meshsetu.site.$siteId');
```

16.3 AEAD envelope

```
final class CipherBlob {
  const CipherBlob({required this.nonce, required this.cipherText, required this.mac});
  final Uint8List nonce, cipherText, mac;
}

final _aes = AesGcm.with256bits();

Future<CipherBlob> encryptAesGcm(SecretKey key, Uint8List plaintext, Uint8List aad) async {
  final box = await _aes.encrypt(plaintext, secretKey: key, aad: aad);
  return CipherBlob(nonce: Uint8List.fromList(box.nonce), cipherText: Uint8List.fromList(box.cipherText), mac:
    Uint8List.fromList(box.mac.bytes));
}

Future<Uint8List> decryptAesGcm(SecretKey key, CipherBlob blob, Uint8List aad) async {
  final clear = await _aes.decrypt(
    SecretBox(blob.cipherText, nonce: blob.nonce, mac: Mac(blob.mac)),
    secretKey: key, aad: aad,
  );
  return Uint8List.fromList(clear);
}
```

16.4 Key hierarchy

```
Authority signing key (offline-managed / production)
|
+-- signs EventManifest
|
+-- Site transport key (demo shared; production provisioned securely)
+-- Public Room key (if confidentiality desired)
+-- Responder Room key (restricted)
+-- Medical Room key (restricted)

Device Keystore key wraps locally cached room/site keys at rest.
```

16.5 Privacy defaults

- Use rotating ephemeral IDs; do not broadcast phone number, name, IMEI or persistent account ID.
- Keep zone-level aggregation rather than user trails for crowd-density features.
- Voice clip has explicit short retention and is not used to train models by default.
- Raw audio should never be uploaded to a cloud service in the offline demo path.
- If optional medical/contact notes are added, encrypt separately and make them consented/role-restricted.
- Log protocol metrics without storing sensitive transcript/audio unless needed for the controlled demo.

17. Persistence, background execution and observability

17.1 Drift database entities

```
class OutboxRows extends Table {
  IntColumn get objectId => integer();
  TextColumn get eventId => text();
  IntColumn get trafficClass => integer();
  BlobColumn get encryptedBlob => blob();
  TextColumn get state => text();
  IntColumn get attempts => integer().withDefault(const Constant(0));
  IntColumn get createdAtMs => integer();
  IntColumn get expiresAtMs => integer();

  @override Set<Column<Object>> get primaryKey => {objectId};
}

Future<List<OutboxRow>> nextOutbox(AppDatabase db, int nowMs, {int limit = 50}) {
  return (db.select(db.outboxRows)
    ..where((t) => (t.state.equals('READY') | t.state.equals('RETRY')) & t.expiresAtMs.isBiggerThanValue(nowMs))
    ..orderBy([(t) => OrderingTerm.asc(t.trafficClass), (t) => OrderingTerm.asc(t.createdAtMs)])
    ..limit(limit))
```

```

    .get();
}

```

17.2 Foreground active-event service

Background BLE is constrained by Android even when the app is written in Flutter. During an active event session, use a user-visible foreground task/service and persistent notification, initiated from Flutter with `flutter\_foreground\_task` or a small platform bridge. Universal BLE documents background scanning patterns; permissions must be granted before the background worker starts. Treat this as an operational mode with explicit user visibility, not an invisible always-on service. [SRC-F04][SRC-F14][SRC-A08]

```

@pragma('vm:entry-point')
void startCallback() {
  FlutterForegroundTask.setTaskHandler(MeshTaskHandler());
}

final class MeshTaskHandler extends TaskHandler {
  @override
  Future<void> onStart(DateTime timestamp, TaskStarter starter) async {
    // Restore durable outbox and restart conservative scan/relay loop.
  }

  @override
  void onRepeatEvent(DateTime timestamp) {
    // Keep work short; actual BLE operations belong to the transport coordinator.
  }
}

```

17.3 Structured metrics

```

final class ProtocolMetric {
  const ProtocolMetric({required this.timeMs, this.eventId, this.peer, required this.kind, this.value, this.detail});
  final int timeMs;
  final String? eventId, peer, detail;
  final String kind; // scan_found, connected, frame_tx, frame_rx, complete, ack, retry...
  final num? value;
}

```

Write metrics as newline-delimited JSON to app-private storage in a debug/hackathon flavor. The log parser can compute delivery probability, median/P95 latency, hops, retransmissions, voice chunk completeness and battery deltas.

17.4 Privacy-safe logs

Never put raw voice bytes, long transcripts, site keys or personal notes into Logcat. Use event IDs and hashed/short peer IDs. A debug-only “show transcript in dashboard” path is separate from protocol telemetry.

18. Testing, benchmarking and failure injection

18.1 Test pyramid

Layer	Tests	Owner
Pure Dart unit	Frame encode/decode, fragmentation/reassembly, TTL, dedupe, priority, rules, schema mapping	Dev A/B
Flutter widget/integration	Drift persistence, permission flow, record smoke, QR scanning, repository/provider wiring	Dev B + STT
Physical radio integration	2-phone GATT, 3-phone relay, reconnect, MTU variation, device churn	Dev A

On-device ML benchmark	STT load/inference, keyword recall, memory; triage class recall	Dev C
System demo	Flutter voice SOS -> 2-hop -> dashboard with internet off	All

18.2 Core unit tests

```
test('fragmentation round trip', () {
  final bytes = Uint8List.fromList(List<int>.generate(10000, (i) => i % 251));
  final frames = fragment(objectId: 42, priority: 1, encrypted: bytes, mtu: 185);
  final buf = ReassemblyBuffer(frames.length, 0);
  for (final f in [...frames].shuffle(Random(7))) { buf.add(f.sequence, f.payload); }
  expect(buf.join(), orderedEquals(bytes));
});

test('structured SOS outranks voice evidence', () {
  final q = <OutboundObject>[];
  void push(OutboundObject o) { q.add(o); q.sort((a,b) => a.trafficClass.rank.compareTo(b.trafficClass.rank)); }
  push(OutboundObject(objectId: 1, trafficClass: TrafficClass.voiceEvidence, createdAtMs: 1, expiresAtMs: 9999,
    bytes: Uint8List(0)));
  push(OutboundObject(objectId: 2, trafficClass: TrafficClass.sosStructured, createdAtMs: 2, expiresAtMs: 9999,
    bytes: Uint8List(0)));
  expect(q.first.objectId, 2);
});
```

18.3 Failure injection checklist

- Disable mobile data and Wi-Fi internet before demo start.
- Force Phone B to be the only path; prove A cannot directly reach gateway by physical separation if possible.
- Kill/restart app on a relay; confirm outbox survives as designed.
- Turn Bluetooth off/on; record recovery time.
- Drop every Nth voice frame in a debug transport wrapper; prove NACK/retry or incomplete state.
- Make STT throw intentionally; prove SOS still reaches dashboard.
- Make triage return low confidence; prove operator sees uncertainty and no SOS suppression.
- Remove a beacon; prove zone result degrades instead of inventing precision.
- Flood 20 Room messages, then generate P0 SOS; prove emergency queue preempts them.

18.4 Metrics the judges should see

Metric	How to measure	Do not claim before measuring
SOS delivery rate	N trials at 1-hop/2-hop with internet off	“99.9% reliable”
Relay latency	source persisted timestamp → gateway complete timestamp	“instant”
P95 latency	at least 20 trials if time permits	single best-case screenshot
Voice encoded size	bytes / seconds for selected codec settings	generic bitrate as product fact
Voice reassembly latency	enqueue → hash-verified complete	live voice capability
STT latency	record end → transcript result	real-time on every phone
STT keyword recall	controlled phrase set	medical-grade ASR
Battery delta	controlled active-event interval	all-day impact without test

19. Complete hackathon build plan

19.1 First principle

The vertical slice comes before feature breadth. By hour 12, the team must have a tiny object relaying from Phone A → Phone B → gateway with internet off, even if the UI is ugly and STT is stubbed. Everything else attaches to that spine.

19.2 48-hour plan

Time	Dev A - Mesh/Protocol	Dev B - Flutter App/Product/Gateway	Dev C - STT/Inference	Integration gate
0-2h	Create Dart protocol/BLE packages, UUIDs, frame contract, universal_ble smoke	Create Flutter shell, Riverpod graph, Drift schema, dashboard skeleton	Prepare 10-20 test clips; build native model candidates / FFI harness	Flutter repo builds on all laptops; shared protobuf/domain contracts frozen.
2-6h	Flutter scan+advertise; 2-phone connect; GATT RX/TX	SOS form + durable outbox; local HTTP dashboard ingest	Run whisper.cpp vs sherpa candidate on target phone; record load/latency/memory	2 phones exchange HELLO and one 100-byte object from Flutter.
6-12h	Fragment/reassemble; dedupe; basic store-forward; 3-phone test	Mesh Code manual entry + Rooms UI; dashboard WebSocket cards	Choose STT engine; implement Dart `OfflineSttEngine`; transcript from PCM	A->B->C object visible in dashboard; STT returns offline transcript through Flutter adapter.
12-18h	Priority scheduler; ACK/retry; MTU handling; protocol metrics	Voice PCM capture with `record`; QR stretch; incident UI	Optimize model threads/quantization; expose timing/model ID; failure fallback	Typed SOS travels 2 hops with measured latency.
18-24h	Voice object transport, NACK/retry or completion ACK	libopus FFI encode + voice manifest + local playback state	Integrate STT with the same PCM capture; test noisy phrases	Transcript + structured SOS + compressed bytes linked to same event.
24-30h	Security envelope + expiry + replay cache	Triage rules/schema; gateway payload; operator ACK UI	Help build/evaluate small triage classifier; STT keyword recall	Structured SOS first, voice second; queue preemption proven.

30-36h	Beacon zone observation; failure injection hooks	Zone/precursor demo UI; QR if not done; polish status screens	Model packaging/warm-up; benchmark report and fallback tests	Full no-internet vertical demo succeeds 5 times consecutively.
36-42h	Radio stability/device churn; metrics parser	Flutter UI polish; demo reset/seed buttons; package/version freeze	STT edge cases; final model checksum/license notes	Freeze features. Only bug fixes after hour 42.
42-48h	Run 20+ trials; fix P0 bugs only	Demo choreography, operator screen, backup APK/server	Produce STT benchmark table; support rehearsals	Final rehearsal with airplane mode, physical phones, timer, backup video/logs.

19.3 If only 24 hours are available

- Keep manual Mesh Code; defer QR scanner.
- Use typed + recorded voice SOS but only one chosen STT model.
- Use completion ACK only; defer missing-chunk bitmap and advanced inventory sync.
- Use nearest-beacon logical zone; simulate density/precursor inputs.
- Use rules-first triage; classifier can be a stretch if training threatens integration.
- Use one laptop dashboard and one gateway phone on local hotspot.

19.4 Build order dependencies

```

1. Domain/schema freeze
  -> 2. Durable outbox
    -> 3. BLE two-phone transfer
      -> 4. 3-phone store-and-forward
        -> 5. Priority + retry + metrics
          -> 6. Voice encoded object
            -> 7. STT transcript attachment
              -> 8. Triage structured SOS
                -> 9. Dashboard + zone intelligence polish

```

QR, advanced localization, density and visual polish must never block steps 1-8.

20. Three-person team split and integration contracts

20.1 Team ownership

Person	Primary ownership	Secondary ownership	Must not become bottleneck for
Developer A - Mesh/Protocol	Flutter/Dart `universal_ble` adapter, peer sessions, frame codec, fragmentation/reassembly, relay, scheduler, ACK/retry, metrics, crypto envelope; native Android bridge only if required	Beacon discovery, radio failure tests	Flutter UI, STT model internals, dashboard styling

Developer B - Flutter App/Product/Integration	Flutter UI, Riverpod, Mesh Code/QR, Rooms, SOS state machine, Drift DB, `record`, Opus FFI orchestration, gateway HTTP, dashboard, demo flow	Triage integration, playback, event/reset tooling	BLE protocol internals, ASR model optimization
Developer C - STT/On-device inference	STT model selection, native C/C++ integration via Dart FFI, preprocessing, quantization/threads, offline benchmarks, model packaging, inference adapter	Triage classifier/data/evaluation, model telemetry	BLE transport and general Flutter UI

20.2 Frozen interfaces at hour 2

```
// Dev A provides to the rest of the Flutter app
abstract interface class MeshTransport {
  Stream<ReceivedObject> get incoming;
  Stream<List<PeerState>> get peerStates;
  Future<SendTicket> send(MeshObject object);
}

// Dev B provides
abstract interface class SosRepository {
  Future<String> createDraft(SosInput input);
  Future<void> attachTranscript(String eventId, SttResult stt);
  Future<void> attachVoice(String eventId, Uint8List encoded);
  Future<void> attachTriage(String eventId, TriageOutput output);
  Future<void> finalizeAndEnqueue(String eventId);
}

// Dev C provides
abstract interface class OfflineSttEngine {
  Future<void> warmUp();
  Future<SttResult> transcribe(Uint8List pcm16le, {int sampleRateHz = 16000});
  Future<void> close();
}
```

20.3 Daily/periodic merge discipline

- No long-lived branches. Merge small vertical commits every 2-3 hours.
- Any contract change requires a 5-minute team sync and compile fix in all dependent modules immediately.
- All three developers keep a “known good demo” git tag/commit after each integration gate.
- No library upgrade after hour 30 unless it fixes a blocking defect.
- Feature flags allow STT, voice, triage, QR and precursor UI to be disabled independently for debugging.

20.4 Developer A checklist

- Two-way scan/advertise works on every demo phone.
- GATT server and client coexist without crashes; peer session state is observable.
- Fragmentation works at multiple synthetic MTUs; voice path has a low-MTU cutoff.
- Dedupe prevents loops; hop/expiry bounds are enforced.
- Priority queue demonstrably preempts voice/chat with SOS.
- Outbox/ACK/retry behavior produces metrics for judges.
- One debug switch can drop/corrupt frames for failure demo.

20.5 Developer B checklist

- Join → Rooms → SOS flow is understandable in under 30 seconds.
- SOS is persisted before model work begins.
- `record` captures 16 kHz mono PCM16 in Flutter; the Opus object is duration-capped and encoded locally before BLE fragmentation.
- Gateway sends verified events to local dashboard without internet.
- Dashboard clearly separates incident priority from zone precursor score.
- Voice state shows queued/transferring/complete/failed; playback appears only after integrity passes.
- Demo reset clears only demo data, not required model assets.

20.6 Developer C checklist

- One STT engine is selected from measured results, not preference.
- Model is fully local after installation; no cloud request occurs during inference.
- Input contract is exactly 16 kHz mono PCM16 or a documented converter exists.
- Warm-up/load/inference time and model size are recorded.
- No fake confidence number is emitted if model cannot support it.
- Critical keyword test set is run and results saved.
- App can continue when `OfflineSttEngine` returns failure.
- Triage classifier (if built) exports fixed labels/version/model hash and never overrides hard safety rules downward.

20.7 Integration handshake schedule

Checkpoint	Required demo
Hour 6	A<->B 100-byte object from Flutter + STT model can transcribe one local sample independently.
Hour 12	A->B->C relay + Dart STT adapter callable from Flutter.
Hour 18	SOS from Flutter UI reaches dashboard; PCM capture and STT linked locally.
Hour 24	Voice is Opus-encoded/chunked across mesh; structured metadata preemption shown.
Hour 30	Triage card + security envelope + dashboard fields integrated.
Hour 36	Full sequence repeated five times with internet off on physical Flutter/Android devices.
Hour 42	Feature freeze; only reliability/presentation work.

21. Demo runbook and definition of done

21.1 Physical setup

- Phone A: citizen sender, microphone permission granted, event manifest loaded.
- Phone B: relay only; physically placed so it is necessary for the path if possible.
- Phone C: gateway/receiver connected to laptop's local hotspot; mobile data disabled.
- Optional Phone D/E: additional relay or beacon simulator.

- Laptop: FastAPI dashboard server; browser full-screen operator view.
- All phones: time approximately synchronized; debug logs enabled; battery >50%.

21.2 90-second technical demo

24. Show mobile data/internet disabled. Start active event mode on all phones.
25. Phone A enters Mesh Code or scans QR and lands in Gate-B/Public Rooms.
26. Hold voice SOS and say a short emergency phrase such as “I cannot breathe, people are pushing near Gate B.”
27. On Phone A show local transcript and Structured SOS card. Point out model/triage are on-device.
28. Show packet animation/status: structured SOS queued at P0/P1; voice evidence below it.
29. Move/position so Phone B relays. Dashboard receives incident, displays hops/latency/transcript/zone.
30. While voice is still transferring, inject another SOS or Room message and show priority ordering.
31. When audio completes, dashboard/gateway shows integrity-verified voice available and plays it.
32. Show zone precursor score separately and state it is advisory, not auto-dispatch.
33. Open metrics panel/log summary: delivery trials, latency, encoded size, STT benchmark.

21.3 Definition of done

Area	Pass
Offline relay	At least 10 consecutive 2-hop structured SOS trials succeed in final setup or failures are quantified and explained.
Priority	New structured SOS overtakes queued voice/Room traffic in a controlled test.
Voice	Short Opus clip is compressed, chunked, reassembled and integrity-verified; transcript arrives first.
STT	Chosen model transcribes locally with internet off; benchmark report includes latency/model ID and phrase-set result.
Triage	Structured fixed-schema card appears with rationale/confidence; forced model error does not suppress SOS.
Join/Rooms	Mesh Code loads correct event/Rooms; restricted role behavior is at least simulated and documented.
Dashboard	Operator sees event, transcript, priority, zone, hops/latency and audio state.
Failure behavior	At least STT failure + relay churn + voice chunk drop are demonstrated/tested.
Claims	Slides/docs say “prototype measured result” for measured figures and avoid standards/certification overclaim.

22. Production hardening roadmap

22.1 After hackathon: protocol

- Replace ad-hoc shared site authentication with signed provisioning and per-role/per-room keys.
- Formalize immutable origin-signed envelope + mutable relay header; fuzz parser and cap allocations.
- Implement inventory summaries and efficient missing-object synchronization rather than opportunistic push alone.
- Test dozens/hundreds of nodes in a simulator and progressively larger controlled RF drills.
- Add embedded relay/beacon firmware and evaluate standards-compliant Bluetooth Mesh / proxy interoperability where it benefits deployment.

22.2 After hackathon: Flutter/Android reliability

- Broaden device/API matrix and document OEM-specific background constraints.
- Profile battery/thermal impact for hours-long event mode.
- Add production-grade Flutter background/event-mode handling: `flutter\_foreground\_task` where appropriate, platform channels/Pigeon for unavoidable Android-only capabilities, device-specific BLE qualification, and a documented power/background policy. Keep Android native code behind interfaces so the Flutter feature layer remains portable. [SRC-F02][SRC-F14]
- Implement robust BLE connection concurrency limits and adaptive scan duty cycles.
- Perform accessibility review and multilingual UI/localization.

22.3 After hackathon: ML

- Create governed, consented emergency-phrases evaluation datasets; stratify by language/noise/device.
- Calibrate STT and triage uncertainty; keep original evidence available to operators.
- Evaluate model update signatures, rollback and offline model-package distribution.
- Validate triage with emergency-domain experts before field use; keep automation advisory unless explicitly authorized.
- Run on-device profiling across CPU/GPU/NPU variants; select runtime/model per device class if needed.

22.4 After hackathon: public-sector integration

- Define signed authority API adapter instead of direct coupling to one emergency system.
- Add deployment manifest tooling: site geometry, beacon IDs, rooms, escalation contacts, model versions and retention policies.
- Create audit/export format for incident timeline and operator actions.
- Complete threat model, privacy impact assessment, DPDP/legal review and retention policy before real-event deployment.
- Run drills with local authorities before any public mass deployment.

Appendix A. Reference code snippets

A.1 Feature flags

```
final class FeatureFlags {  
  const FeatureFlags({this.qrJoin = true, this.voice = true, this.stt = true, this.triageModel = true,  
    this.beaconZone = true, this.precursor = true});  
  final bool qrJoin, voice, stt, triageModel, beaconZone, precursor;  
}
```

A.2 SOS orchestration use case

```
final class SubmitVoiceSos {  
  const SubmitVoiceSos(this.repo, this.stt, this.encoder, this.triage);  
  final SosRepository repo;  
  final OfflineSttEngine stt;  
  final VoiceEncoder encoder;  
}
```

```

final TriageEngineImpl triage;

Future<String> call(SosInput input, Uint8List pcm16le) async {
  final eventId = await repo.createDraft(input);
  final sttFuture = stt.transcribe(pcm16le);
  final voiceFuture = encoder.encodeOpus(pcm16le);

  SttResult? sttResult;
  try { sttResult = await sttFuture; await repo.attachTranscript(eventId, sttResult); } catch (_) { /* raw SOS
still continues */ }

  final structured = await triage.triage(sttResult?.text ?? '');
  await repo.attachTriage(eventId, structured);

  final encoded = await voiceFuture;
  await repo.attachVoice(eventId, encoded);
  await repo.finalizeAndEnqueue(eventId);
  return eventId;
}
}

```

If STT fails, the orchestration must still attach the raw voice evidence and a fallback structured record such as `incidentType=OTHER`, `priority=P1\_HIGH` for an explicit emergency button, with `missingInfo=[transcript]`. Product policy should decide the safe fallback band.

A.3 Priority-aware per-peer sender loop

```

Future<void> sendLoop(PeerSession peer) async {
  while (!peer.closed) {
    final next = scheduler.nextFor(peer.peerId);
    if (next == null) { await Future<void>.delayed(const Duration(milliseconds: 80)); continue; }
    final frames = fragment(objectId: next.objectId, priority: next.trafficClass.rank, encrypted: next.bytes, mtu:
peer.mtu);
    for (final frame in frames) {
      if (scheduler.hasHigherPriorityThan(next.trafficClass)) break;
      await writeFrame(peer, frameCodec.encode(frame));
    }
  }
}

```

A.4 Missing chunk bitmap

```

Uint8List missingBitmap(int count, List<bool> received) {
  final out = Uint8List((count + 7) ~/ 8);
  for (var i = 0; i < count; i++) { if (!received[i]) out[i ~/ 8] |= 1 << (i % 8); }
  return out;
}

```

A.5 Site fingerprint

```

Future<Uint8List> siteFingerprint(String siteId, String namespace) async {
  final digest = await Sha256().hash(utf8.encode('$siteId|$namespace'));
  return Uint8List.fromList(digest.bytes.take(6).toList()); // discovery hint only; not authentication
}

```

A.6 RSSI debug panel model

```

final class PeerDebug {
  const PeerDebug({required this.id, required this.rssi, required this.mtu, required this.connected, required
this.queuedObjects, required this.lastSeenMs});
  final String id;
  final int rssi, mtu, queuedObjects, lastSeenMs;
  final bool connected;
}

```

A.7 Trace timing

```

final class TraceStamp {
  const TraceStamp(this.name, this.elapsedMs);
  final String name;
  final int elapsedMs;
}

final class EventTrace {
  EventTrace(this.eventId) : _clock = Stopwatch()..start();
  final String eventId;
  final Stopwatch _clock;
  final List<TraceStamp> _stamps = [];
  void mark(String name) => _stamps.add(TraceStamp(name, _clock.elapsedMilliseconds));
  int? delta(String from, String to) {
    final fromMatches = _stamps.where((s) => s.name == from);
    final toMatches = _stamps.where((s) => s.name == to);
    if (fromMatches.isEmpty || toMatches.isEmpty) return null;
  }
}

```

```

    return toMatches.first.elapsedMs - fromMatches.first.elapsedMs;
}
}

```

A.8 Python log summary

```

# tools/log_parser.py
import json, statistics, sys
rows=[json.loads(x) for x in open(sys.argv[1], encoding='utf8') if x.strip()]
lat=[r['value'] for r in rows if r.get('kind')=='relay_latency_ms' and r.get('value') is not None]
print('trials', len(lat))
if lat:
    print('median_ms', statistics.median(lat))
    print('max_ms', max(lat))
    if len(lat)>=20:
        s=sorted(lat); print('p95_ms', s[int(.95*(len(s)-1))])

```

A.9 Triage test examples

```

final cases = <(String, PriorityBand)>[
    ('I cannot breathe, people are pushing near Gate B', PriorityBand.p0Critical),
    ('A person is unconscious near the stairs', PriorityBand.p0Critical),
    ('Small cut, need first aid', PriorityBand.p1High),
];

```

A.10 Debug frame-loss interceptor

```

final class LossyTransport implements FrameTransport {
    LossyTransport(this.delegate, this.dropEvery);
    final FrameTransport delegate;
    final int dropEvery;
    int _n = 0;

    @override
    Future<bool> send(Uint8List frame) async {
        _n++;
        if (dropEvery > 0 && _n % dropEvery == 0) return true; // simulated loss
        return delegate.send(frame);
    }
}

```

Appendix B. Command and debugging cheat sheet

B.1 Flutter / Android build

```

flutter doctor -v
flutter devices
flutter pub get
dart run build_runner build --delete-conflicting-outputs
flutter analyze
flutter test
flutter run -d <DEVICE_ID>
flutter build apk --debug
adb logcat | grep -i meshsetu

```

B.2 Dashboard

```

cd dashboard
python -m venv .venv
# macOS/Linux: source .venv/bin/activate
# Windows: .venv\Scripts\activate
pip install fastapi "uvicorn[standard]"
uvicorn main:app --host 0.0.0.0 --port 8080 --reload

```

B.3 Network check

```

# Laptop local IP (examples only)
ipconfig          # Windows
ifconfig          # macOS/Linux

# From the Flutter Android device connected to the same local LAN/hotspot:
http://<LAPTOP_LAN_IP>:8080

```

B.4 BLE debugging reminders

- Keep screen/dev debug panel showing Bluetooth enabled, advertising, scanning, peers, GATT state, MTU and queue length.

- If connect fails: log the `universal\_ble`/platform error, disconnect the peer through the Flutter transport adapter, back off, rediscover, and retry. Keep Android-native GATT details behind the adapter unless a platform-specific fix is required.
- If frames stall: check write-without-response pacing; some stacks require a small credit/delay strategy.
- If voice destroys latency: lower bitrate/duration, reduce fan-out, increase SOS preemption, or disable voice on low-MTU peers.
- If relay loops: inspect dedupe cache, event/object ID generation and whether re-encryption creates a new object ID each hop. Preserve origin object identity across hops.

B.5 STT debugging

- Always log model ID/hash, load ms, inference ms, PCM sample count, sample rate and thread count.
- Validate PCM amplitude and duration before blaming the model.
- Whisper-style models generally expect 16 kHz audio; resample explicitly if source differs.
- Benchmark airplane mode to prove local inference.
- Thermal throttling can make repeated inference slower; include warm/cold and repeated runs.

Appendix C. Official documentation index

Use these links as the source of truth while implementing. The architecture deliberately relies on primary/official documentation rather than random tutorials. Versions/APIs can change after this document date.

[SRC-F01] Flutter documentation: <https://docs.flutter.dev/>

[SRC-F02] Flutter platform channels / platform-specific code:
<https://docs.flutter.dev/platform-integration/platform-channels>

[SRC-F03] Dart native C interoperability (dart:ffi): <https://dart.dev/interop/c-interop>

[SRC-F04] universal_ble Flutter package (central + peripheral BLE): https://pub.dev/packages/universal_ble

[SRC-F05] record Flutter package (PCM stream / recording): <https://pub.dev/packages/record>

[SRC-F06] Drift reactive SQLite persistence: <https://pub.dev/packages/drift>

[SRC-F07] drift_flutter helper: https://pub.dev/packages/drift_flutter

[SRC-F08] Riverpod documentation: <https://riverpod.dev/>

[SRC-F09] mobile_scanner Flutter package: https://pub.dev/packages/mobile_scanner

[SRC-F10] just_audio Flutter package: https://pub.dev/packages/just_audio

[SRC-F11] tflite_flutter package: https://pub.dev/packages/tflite_flutter

[SRC-F12] flutter_secure_storage package: https://pub.dev/packages/flutter_secure_storage

[SRC-F13] cryptography Dart package: <https://pub.dev/packages/cryptography>

[SRC-F14] flutter_foreground_task package: https://pub.dev/packages/flutter_foreground_task

[SRC-F15] Dart HTTP package: <https://pub.dev/packages/http>

[SRC-B01] Bluetooth SIG Mesh FAQ / concepts: <https://www.bluetooth.com/learn-about-bluetooth/feature-enhancements/mesh/>

[SRC-B02] Bluetooth Mesh Protocol specification landing page:
<https://www.bluetooth.com/specifications/specs/mesh-protocol/>

[SRC-O01] Opus codec official site: <https://opus-codec.org/>

[SRC-O02] IETF/RFC 6716 - Opus codec: <https://www.rfc-editor.org/rfc/rfc6716>

[SRC-O03] libopus encoder API: https://opus-codec.org/docs/opus_api-1.6/group__opus__encoder.html

[SRC-S01] whisper.cpp official GitHub: <https://github.com/ggml-org/whisper.cpp>

[SRC-S02] sherpa-onnx official documentation: <https://k2-fsa.github.io/sherpa/onnx/>

[SRC-M01] LiteRT overview: <https://ai.google.dev/edge/litert>

[SRC-M02] TensorFlow Lite Flutter plugin documentation: https://pub.dev/packages/tflite_flutter

[SRC-M03] ONNX Runtime mobile documentation: <https://onnxruntime.ai/docs/tutorials/mobile/>

[SRC-P01] Protocol Buffers Dart runtime: <https://pub.dev/packages/protobuf>

[SRC-P02] protoc_plugin Dart code generator: https://pub.dev/packages/protoc_plugin

[SRC-A03] Android BluetoothGatt / MTU API (underlying Android behavior):
<https://developer.android.com/reference/android/bluetooth/BluetoothGatt>

[SRC-A04] Android Bluetooth permissions: <https://developer.android.com/develop/connectivity/bluetooth/bt-permissions>

[SRC-A08] Android foreground services / connected-device type:
<https://developer.android.com/develop/background-work/services/fgs/service-types>

[SRC-W01] FastAPI WebSockets: <https://fastapi.tiangolo.com/advanced/websockets/>

C.1 Foundational MeshSetu documents

The original MeshSetu research paper defines the five-layer architecture, multi-hop SOS, beacon-referenced localization, privacy-preserving density, precursor scoring, authority bridge, evaluation metrics and human-in-the-loop constraints. The later product-content bible adds Mesh Code, Rooms, local Speech-to-Text + compressed voice-note relay and on-device triage. Treat the research paper as architectural foundation and this technical bible as the implementation contract for the MVP.

Local project source files used while preparing this document: `MeshSetu Research Paper.pdf` and `MeshSetu\_PPT\_Master\_Content\_Bible\_Research\_Evidence\_Updated.docx`.

Final engineering checklist

- ☐ One README command sequence gets the Flutter app building on all developer machines with the same pinned Flutter SDK and `pubspec.lock`.
- ☐ Physical BLE 2-hop vertical slice works from the Flutter app before visual polish.
- ☐ No Text-to-Speech code, UI or claim exists; Speech-to-Text is the only speech-model feature.
- ☐ Voice clip is bounded, compressed, chunked and store-and-forward; no live stream claim.
- ☐ Offline STT model is packaged behind a Dart interface/FFI adapter, tested with internet off, and has a measured benchmark on real phones.
- ☐ Structured SOS is durable and prioritized ahead of voice/chat.
- ☐ Triage is schema-constrained, conservative and cannot suppress a user-triggered SOS.
- ☐ Mesh Code is bootstrap/namespace UX, not a short cryptographic key.
- ☐ Rooms have ACL/priority/TTL policy; responder traffic is not exposed to public users in production design.
- ☐ Every object has ID, creation/expiry, site/room scope and integrity protection.
- ☐ Dedupe/TTL prevents infinite relay loops.
- ☐ Low MTU / STT failure / audio failure / relay churn all have visible graceful-degradation behavior.
- ☐ Dashboard works on local LAN with internet disabled.
- ☐ Final demo report includes actual delivery, latency, voice-size and STT measurements.
- ☐ Unknown fourth product feature remains a documented extension point, not an invented claim.